

Lecture Script

Attention and Transformers

Outline

- 1 Data-independent weights
- 2 Building up attention
- 3 The transformer block
- 4 The residual stream
- 5 Multi-head attention
- 6 Positional information

1. Data-independent weights

[10 min]

An MLP layer with $x \in \mathbb{R}^d$, $W \in \mathbb{R}^{m \times d}$:

$$y = \sigma(Wx + b)$$

W is **fixed after training**. Every input gets multiplied by the same W —**data-independent** weights.

What does W actually do?

Row i of W computes $y_i = w_i^\top x$, where $w_i \in \mathbb{R}^d$. A fixed dot product—a fixed linear measurement of x . The input has no say in *what gets measured*.

When is this limiting?

You have n vectors $x_1, \dots, x_n \in \mathbb{R}^d$. Two obvious approaches:

- Concatenate: $y = \sigma(W[x_1; \dots; x_n])$ — but $W \in \mathbb{R}^{d \times nd}$, so it breaks for different n .
- Shared MLP: $y_i = \sigma(Wx_i)$ — works for any n , but $y_i = f(x_i)$. Each element is computed in isolation; no mechanism for x_i to know what else is in the set.

We want $y_i = f(x_i, \{x_j\})$: each element's representation depends on the rest of the set. How?

Aggregate from all elements, weighted by relevance:

$$y_i = \sum_{j=1}^n \alpha_{ij} x_j$$

where α_{ij} are not learned—they're computed from the inputs. We'll build up α_{ij} in stages.

2. Building up attention

[25 min]

Step 1: Raw dot-product similarity (nothing learned)

Weight by similarity. Similarity = dot product, softmax to normalize:

$$\alpha_{ij} = \frac{\exp(x_i^\top x_j)}{\sum_{k=1}^n \exp(x_i^\top x_k)}$$

$\alpha_{ij} \geq 0$, $\sum_j \alpha_{ij} = 1$. Element i pulls more from elements similar to it. No learned parameters.

In matrix form, stacking inputs as rows of $X \in \mathbb{R}^{n \times d}$:

$$Y = \underbrace{\text{softmax}(XX^\top)}_{\text{weights} \in \mathbb{R}^{n \times n}} \underbrace{X}_{\text{values to mix}}$$

$XX^T \in \mathbb{R}^{n \times n}$ is pairwise similarity; softmax normalizes each row. Compare to $Y = \sigma(XW)$: here $\text{softmax}(XX^T)$ plays the role of a mixing matrix, but it's computed from the input.

Note what W does vs. what attention does: W *transforms features* (same for every element). Attention *mixes elements* (data-dependent). Different jobs.

Step 2: Learned metric (one matrix)

Project before computing the score:

$$s_{ij} = (Wx_i)^T (Wx_j) = x_i^T \underbrace{W^T W}_{\text{learned metric}} x_j$$

This learns a Mahalanobis-like metric—reshapes what “similar” means. But query and key are still the same function of x , and we still return row x_j .

Step 3: Separate query and key (two matrices)

$$s_{ij} = (W_Q x_i)^T (W_K x_j)$$

Now “what am I looking for?” ($W_Q x_i$) is decoupled from “what do I advertise?” ($W_K x_j$). The matching is asymmetric—an element can look for one pattern while broadcasting a different one. But still returning row x_j .

Step 4: Separate values (three matrices — full attention)

$$y_i = \sum_j \text{softmax}_j \left(\frac{(W_Q x_i)^T (W_K x_j)}{\sqrt{d_k}} \right) W_V x_j$$

Three decoupled roles:

- **Query** $q_i = W_Q x_i \in \mathbb{R}^{d_k}$: what element i is looking for

- **Key** $k_j = W_K x_j \in \mathbb{R}^{d_k}$: what element j advertises
- **Value** $v_j = W_V x_j \in \mathbb{R}^{d_v}$: what element j hands over when selected

In matrix form, stacking n elements as rows of $X \in \mathbb{R}^{n \times d}$ (so projections multiply on the right: $Q = XW_Q$, etc.):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The $\sqrt{d_k}$: dot products grow with dimension, pushing softmax toward one-hot. Dividing by $\sqrt{d_k}$ keeps gradients stable.

The progression: raw dot product $\rightarrow$ learned metric $\rightarrow$ asymmetric matching $\rightarrow$ asymmetric matching with decoupled retrieval. Each step adds a degree of freedom in how elements talk to each other.

The attention matrix

$A = \text{softmax}(QK^\top / \sqrt{d_k}) \in \mathbb{R}^{n \times n}$. Row i is a distribution over elements—the attention pattern for i . Not symmetric.

Attention as set operation

Permutation equivariant: permute rows of X , rows/columns of $QK^\top$ permute consistently, output permutes the same way. No assumption of sequence or order.

3. The transformer block

[10 min]

What attention can't do

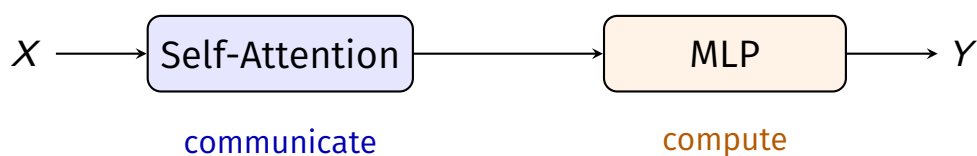
$y_i = \sum_j \alpha_{ij} v_j$ is a **weighted average** of value vectors. Attention is nonlinear overall (the softmax makes α_{ij} a nonlinear function of the input), but *for a given set of weights*, y_i is just a convex combination of $\{v_j\}$. There's no elementwise nonlinearity applied to the result—no capacity to transform what was gathered.

Adding an MLP

Follow attention with an MLP applied independently per element. With $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, $W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$:

$$\text{MLP}(y_i) = W_2 \sigma(W_1 y_i + b_1) + b_2$$

The **transformer block**:



Attention = *inter*-element (communicate). MLP = *intra*-element (compute).

► Now stack these blocks.

4. The residual stream

[10 min]

Residual connections

Stacking blocks without skip connections makes it hard to preserve earlier representations, and gradients tend to vanish through many layers.

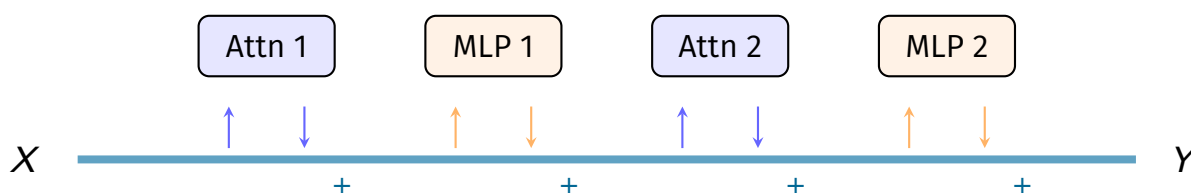
Residual connections (from ResNets): $y = x + f(x)$. The layer learns the *change*, not the whole representation. Applied around both sub-layers:

$$\begin{aligned} X' &= X + \text{Attention}(X) \\ X'' &= X' + \text{MLP}(X') \end{aligned}$$

(Also: layer normalization before each sub-layer, the “pre-norm” convention. Stabilizes training.)

The stream picture

Residual stream: each layer reads from and writes back to a shared representation.



Information accumulates—layer 5 can still read what layer 1 wrote, or the original input embedding.

► What if one attention pattern isn't enough?

5. Multi-head attention

[10 min]

The limitation of a single head

One head = one $n \times n$ attention matrix. A single pattern has to compromise between different reasons to attend.

Multiple heads in parallel

Run h **attention heads in parallel**, each with its own projections in dimension $d_k = d/h$:

$$Q^{(i)} = XW_Q^{(i)}, \quad K^{(i)} = XW_K^{(i)}, \quad V^{(i)} = XW_V^{(i)}$$

Concatenate and project back:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

where $W_O \in \mathbb{R}^{d \times d}$.

Each head operates in $\mathbb{R}^{d/h}$. Total parameters: same as one full-dimension head—a **repartitioning**, not an expansion. Payoff: h different $n \times n$ attention matrices.

► One thing missing: order.

6. Positional information

[10 min]

The symmetry we built in

Everything so far is permutation equivariant. For **sequences**, order matters: “the cat sat on the mat” $\neq$ “the mat sat on the cat.”

Breaking the symmetry

Add positional information to the input embeddings before the transformer.

Given input embeddings $x_1, \dots, x_n$ and position encodings $p_1, \dots, p_n$:

$$\tilde{x}_i = x_i + p_i$$

Permuting the inputs now changes the representation—symmetry broken.

What goes into p_i ?

Sinusoidal (original). The original transformer used fixed sinusoidal functions at different frequencies:

$$p_{i,2k} = \sin(i/10000^{2k/d}), \quad p_{i,2k+1} = \cos(i/10000^{2k/d})$$

Each dimension oscillates at a different frequency—coarse to fine position. No learned parameters.

Learned. $p_1, \dots, p_{n_{\max}}$ as a lookup table of learned vectors. Can't extrapolate beyond $n_{\max}$.

The design choice

Positional encoding is a **modular choice**: sets don't need it, sequences do. The transformer is a set-processing architecture by default.